

Placement Empowerment Program
Cloud Computing and DevOps Centre

Set Up a Local Git Repository: Initialize a Git repository locally and version control your static website

Name: Neithal Pillai

Department : IT

Introduction

Version control is an essential aspect of software development that enables you to manage modifications to your code over time. It provides a structured approach to tracking changes, facilitating collaboration, and reverting to previous versions when necessary. Git is one of the most popular version control systems, valued for its speed, flexibility, and distributed architecture.

In this Proof of Concept (POC), we will initialize a local Git repository to manage version control for your static website. This setup will allow you to monitor project changes, experiment with new features safely, and effortlessly share your project if needed. Establishing a Git repository is a crucial step toward maintaining an organized and efficient workflow, particularly for developers and teams working collaboratively.

Overview

This setup will guide you through the following steps:

1. **Installing Git** – Ensure Git is installed on your system and properly configured.
2. **Creating a Local Repository** – Initialize a Git repository in the root directory of your static website.
3. **Staging and Committing Files** – Add project files to the staging area and commit them to record a snapshot of your work.
4. **Reviewing the Repository State** – Use Git commands to inspect the repository's status and confirm that all files are tracked correctly.

Objectives

By completing this POC, you will:

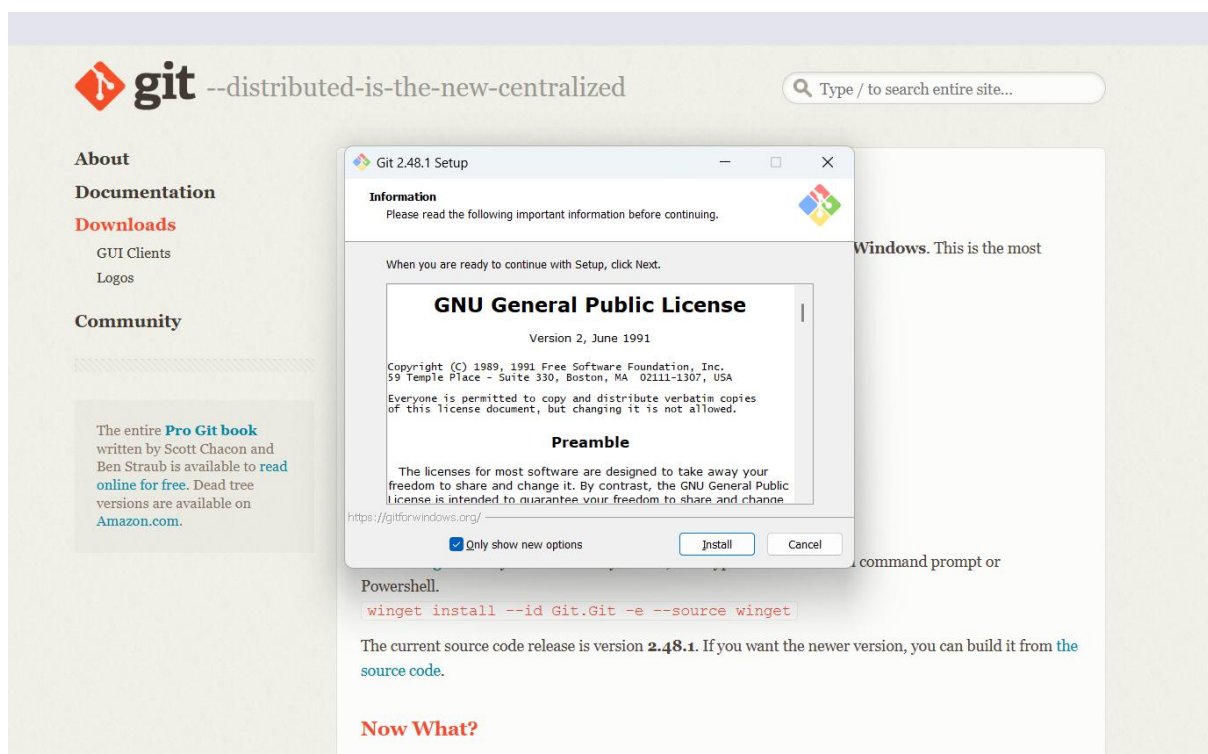
1. **Grasp the Fundamentals of Version Control** – Gain a clear understanding of Git's role in managing and tracking changes within projects.
2. **Initialize a Git Repository** – Learn how to set up a Git repository for locally version-controlling your static website.
3. **Track Modifications Efficiently** – Understand how to stage and commit files, ensuring that every change is properly logged.

4. **Maintain a Well-Structured Project** – Develop an organized workflow that enables easy rollback of changes when needed.
5. **Prepare for Collaboration** – Establish a foundation for sharing your repository and collaborating with others using Git when required.

Steps For Execution:

1. Installed and Configured Git

I started by installing Git on my system and ensuring it was properly set up. After installation, I configured my global username and email so that my commits would be associated with my identity. This setup was important because Git tracks changes made by different users, and having the correct credentials ensures that my contributions are correctly attributed.



2. Initialized a Git Repository

Once Git was set up, I navigated to my project folder, "**POC PROJECTS**," which was located on my Desktop. I initialized it as

a Git repository, which allowed Git to start tracking all the changes made within this folder. This process created a hidden `.git` directory, where Git stores all version control metadata, enabling me to track modifications, roll back changes if necessary, and manage different versions of my project efficiently.



Sign in to GitHub
to continue to **Git Credential Manager**

Username or email address

Password [Forgot password?](#)

Sign in

[Sign in with a passkey](#)

New to GitHub? [Create an account](#)

3. Added and Committed My First File

- a. To begin version controlling my project, I created an `index.html` file, which contained the basic structure of my website. Before making a commit, I checked the status of my repository to confirm that `index.html` was recognized as an untracked file. I then added the file to the staging area, preparing it for a commit. After that, I committed the file with a meaningful message, officially recording the changes in my local repository. At this point, my changes were saved, but they were only available locally and not yet stored on GitHub.

4. Linked My Local Repository to GitHub

- a. In order to store my project online and collaborate with others, I created a new repository on GitHub called "**POC-Projects.**" I then connected my local Git repository to this GitHub repository by adding it as a remote origin. This step was crucial because it allowed me to push my local commits to GitHub and keep my project backed up in a cloud-based version control system.



Authentication Succeeded

You may now close this tab and return to the application.

5. Pushed My First Commit to GitHub

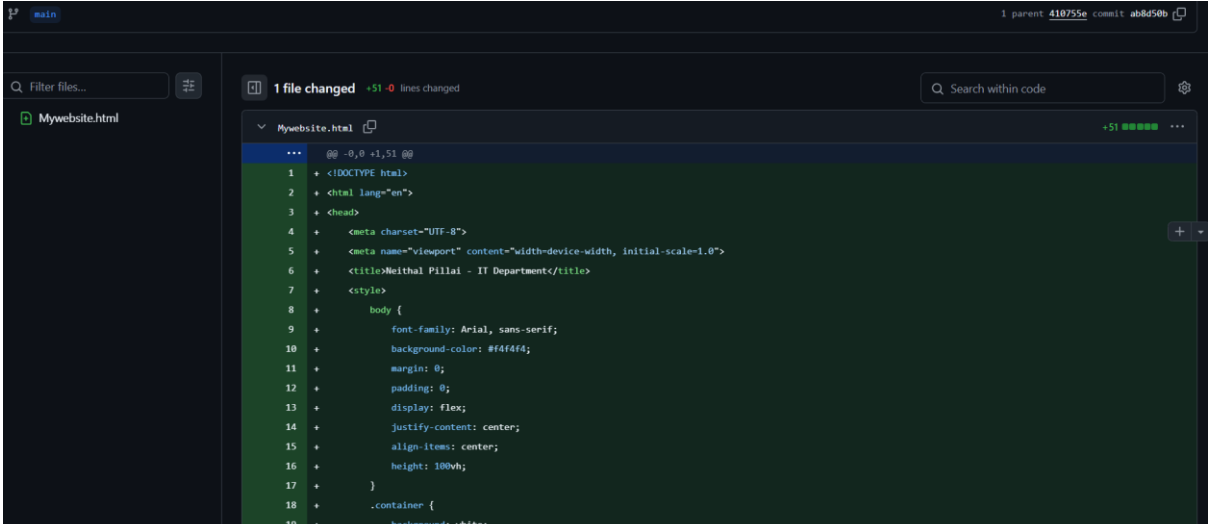
After linking my repository, I pushed my first commit to GitHub. This step uploaded my locally committed changes to the remote repository, making my project accessible online. By doing this, I ensured that my work was safely stored and could be accessed from anywhere, providing an additional layer of security and collaboration opportunities.

6. Added Another File to Git and Pushed It

After successfully pushing my first file, I decided to add another file to my repository. I created a new file in the same folder and wanted Git to track it as well. To do this, I repeated the process: I checked the status of my repository, added the new file to the staging area, committed it with a meaningful message, and then pushed it to GitHub. This ensured that my repository remained up to date with the latest changes.

7. Fixed a Non-Fast-Forward Error

At one point, when I attempted to push my latest changes, I encountered an error stating that my branch was behind the remote repository. This happened because changes had been made in the remote repository that were not present in my local version. To resolve this issue, I first pulled the latest changes from GitHub and merged them with my local branch. After ensuring there were no conflicts, I committed the merged version and then successfully pushed my updated repository back to GitHub.



The screenshot shows a code editor interface with a dark theme. On the left, a sidebar displays a file explorer with 'Mywebsite.html' selected. The main editor area shows the content of 'Mywebsite.html', which is an HTML document with a CSS style block. The code is as follows:

```
@@ -0,0 +1,51 @@
1 + <!DOCTYPE html>
2 + <html lang="en">
3 + <head>
4 +   <meta charset="UTF-8">
5 +   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6 +   <title>Neithal Pillai - IT Department</title>
7 +   <style>
8 +     body {
9 +       font-family: Arial, sans-serif;
10 +      background-color: #f4f4f4;
11 +      margin: 0;
12 +      padding: 0;
13 +      display: flex;
14 +      justify-content: center;
15 +      align-items: center;
16 +      height: 100vh;
17 +    }
18 +    .container {
19 +      background: white;
```

